

Sampling from Your Language Model One Byte at a Time

Jonathan Hayase¹ Alisa Liu¹ Noah A. Smith^{1,2} Sewoong Oh¹

Abstract

Tokenization is used almost universally by modern language models, enabling efficient text representation using multi-byte or multi-character tokens. However, prior work has shown that tokenization can introduce distortion into the model’s generations, an issue known as the *Prompt Boundary Problem (PBP)*. For example, users are often advised not to end their prompts with a space because it prevents the model from including the space as part of the next token. While this heuristic is effective in English, the underlying PBP continues to affect code generation and languages such as Chinese, where tokens often do not line up with word and syntactic boundaries. In this work, we present an inference-time method to convert any autoregressive LM with a BPE tokenizer into a character-level or byte-level LM. Our method efficiently solves the PBP and is also able to unify the vocabularies of language models with different tokenizers, allowing one to ensemble LMs with different tokenizers at inference time or transfer the post-training from one model to another using proxy-tuning.

1. Introduction

Tokenization is a crucial component of modern language models: it allows them to efficiently consume and produce arbitrary streams of text using finite vocabularies. The vast majority of tokenizers in use today, such as those based on Byte-Pair Encoding (BPE) (Sennrich et al., 2016) or Unigram (Kudo & Richardson, 2018), feature tokens spanning multiple bytes or characters, allowing them to represent text more efficiently than purely byte-level or character-level tokenization (Clark et al., 2022; Xue et al., 2022; Wang et al., 2024). Users of LMs are generally unaware of the tokenization and expect LMs to operate on strings over an

¹University of Washington ²Allen Institute for AI. Correspondence to: Jonathan Hayase <jhayase@cs.washington.edu>.

```
> olmo.generate(tok.encode("This a tes"))
"erstor" ✗
> ByteSampler(olmo, "This is a tes")
"t" ✓

> qwen.generate(tok.encode("日本的首都是东京, 中国的首都"))
"也是北京" ✗
> ByteSampler(qwen, "日本的首都是东京, 中国的首都")
"是 北京" ✓

> olmo.generate(tok.encode("document.getElementById"))
"('div')" ✗
> ByteSampler(olmo, "document.getElementById")
"ById('button')" ✓
```

Figure 1. ByteSampler resolves the prompt boundary problem (exhibited in the output of `generate()`). In this example, `_test`, `都是`, and `.getElementById` are all single tokens in the respective tokenizers.

alphabet Σ (e.g. bytes), consuming a `prompt` $\in \Sigma^*$ as a string and producing a string `completion` $\in \Sigma^*$ thereof.

Tokenized LMs emulate this by (i) encoding the `prompt` into a sequence of tokens $(t_1, \dots, t_k) = \text{encode}(\text{prompt})$, (ii) sampling a continuation as a token sequence $t_{k+1}, \dots, t_n$ from the LM using

$$\begin{aligned} &P(t_{k+1}, \dots, t_n \mid t_1, \dots, t_k) \\ &= \prod_{i=k+1}^n P(t_i \mid t_1, \dots, t_{i-1}), \end{aligned} \quad (1)$$

and (iii) decoding the generated token sequence back into text (`completion` $\leftarrow \text{decode}(t_{k+1}, \dots, t_n)$).¹

While sampling this way is easy, it can also lead to distorted predictions due to the conversion to and from token space (Lundberg, 2023; Vieira et al., 2024; Phan et al., 2024).

The Prompt Boundary Problem (PBP). In the above procedure, the final token sequence can be written as $T_{\text{concat}} := \text{encode}(\text{prompt}) \uplus \text{encode}(\text{completion})$ (where $\uplus$ denotes concatenation). This means that, by construction, no token can cross the *boundary* between the `prompt` and `completion`, as the end of the prompt forces a token boundary there. The *prompt boundary problem* then arises when the tokenization of the combined text,

¹P represents the token-level distribution of the model and `encode`: $\Sigma^* \rightarrow V^*$ and `decode`: $V^* \rightarrow \Sigma^*$ translate between strings and token sequences over vocabulary V .

encode(prompt + completion), differs from the concatenated token sequence, T_{concat} . This is problematic because LMs are trained on the output of encode; among all the possible token sequences that represent the same text, LMs are trained on only one of them! This effectively means LMs are trained *not* to produce completion given prompt (and crucially, this has nothing to do with whether or not completion is actually a reasonable continuation of prompt).

As a concrete example, consider OLMo2-1B and suppose the user’s prompt is “Hello wor” ([“Hello” (9906), “_wor” (4191)] as tokens): Given the context, the user likely expects the next token to be “ld” (509). However, OLMo2 has never seen “_world” tokenized as the sequence [“_wor”, “ld”]² and thus assigns $P(\text{“ld”} \mid [\text{“Hello”, “_wor”}]) \approx 5 \times 10^{-4}$. On the other hand, $P(\text{“_world”} \mid [\text{“Hello”}]) = 3 \times 10^{-2}$ — so giving less information in the prompt actually makes the desired string more likely!³ This phenomenon can be made more precise using the notion of token-sequence validity (Phan et al., 2024). In the example above, [“Hello”, “_wor”, “ld”] is invalid while [“Hello”, “_world”] is valid, even though both decode to “Hello world”. We formalize this in Definition 3.3.

While this example may seem contrived, there are many situations where this problem arises naturally (Fig. 1 shows a few more examples):

1. In languages that do not separate words with whitespace, such as Chinese and Japanese, tokens can span multiple words, so this issue can arise even when the prompt ends with a complete word (Xu et al., 2026).
2. Any tokenizer that features multi-word tokens, which can bring gains in encoding efficiency (Gee et al., 2023; Kumar & Thawani, 2022; Liu et al., 2025; Tănase & Pelican, 2025), suffers from the same problem as tokenizers for Chinese and Japanese.
3. When completing code, it is common to request completions while in the middle of an identifier (Jackson, 2025; Bavarian et al., 2022).
4. This issue also occurs when performing constrained generation from language models (Ribeiro, 2023; Beurer-Kellner et al., 2024).

Contributions. In this paper, we propose ByteSampler, a system that can condition LMs on arbitrary *byte-prefixes*.

²In fact, as we will show (in Section 3.4), the tokenizer will never output token 509 following token 4191.

³If we sample two more tokens greedily from the [“Hello”, “_wor”] prefix, we get [“l” (75, $p = 0.2$), “,” (11, $p = 0.1$)], which is probably not what the user wanted.

This can be used to solve the PBP and can also be applied to convert the (tokenized) LM into a byte-level LM. Compared to prior work (Table 1), our method is the first to simultaneously achieve the following objectives:

1. **Exact.** Our method preserves the model’s output distribution, up to probability mass on invalid token sequences. We empirically show that our method preserves language modeling loss in Section 4.2 and preserves utility in downstream tasks (Sections E.5 and F).
2. **Efficient.** Our method is faster and uses fewer inference tokens than all methods of comparable quality (Sections 4.1, E.1 and E.2)
3. **Compatible.** Our method supports BPE tokenizers with future-dependent pretokenization, making it applicable to the vast majority of current open-weight LMs. (Table 1 and Section C.7)

2. Background

In this section we give essential background regarding tokenization as well as prior work addressing the Prompt Boundary Problem. We discuss additional related works in Section A.

Pretokenization and Byte Pair Encoding. BPE was originally presented as a form of data compression in Gage (1994) and was proposed for use in NLP in Sennrich et al. (2016). However, modern tokenizers augment BPE with a *pretokenization* stage, which splits the input text into chunks, called *pretokens*. We show the combined pretokenization and BPE inference in Algorithm 1.

Algorithm 1: BPE with pretokenization

Input: byte string x , ordered merge list $\mathcal{M}$
Output: token sequence T
 $[x_1, \dots, x_m] \leftarrow \text{PRETOKENIZE}(x);$
 $T \leftarrow [];$
foreach pretoken x_i **do**
 $T_i \leftarrow \text{bytes of } x_i;$
 foreach $(t_l, t_r, t_{lr}) \in \mathcal{M}$ **do**
 while there exists adjacent t_l, t_r in T_i **do**
 merge the first match t_l, t_r in T_i into $t_{lr};$
 $T \leftarrow T \# T_i;$
return $T;$

As a result, no token may cross the boundary between pretokens. In some cases, pretokenization boundaries may depend on future bytes, so we use a branching model to handle all possible ambiguities. We discuss this in Section C.3.

Table 1. Comparison of various mitigations for the prompt boundary problem: we list tokenizers supported (S for SentencePiece BPE and H for HuggingFace ByteLevel BPE, see Section C.7 for details and exceptions) and complexity (for both CPU and GPU) when sampling each new character while generating an n character string. Our method has the same complexity as backtracking methods (Ribeiro, 2023; Dagan et al., 2024; Athiwaratkun et al., 2024) while remaining exact. The * indicates that exactness is modulo probability on invalid token sequences (see Section 2).

	Tokenizers	Exact	Orchestration (CPU)	Inference tokens (GPU)
Backtracking	Any	No	$O(1)$	$O(1)$
K -Beam Summing (Vieira et al., 2024)	Any	No	$O(Kn)$	$O(Kn)$
Prefix Covering (Vieira et al., 2024)	Any	Yes	$2^{O(n)}$	$2^{O(n)}$
Back Tokenization (Turaga, 2025)	S	Yes*	$O(n)$	$O(1)$
Exact Byte-Level (Phan et al., 2025)	S	Yes*	$O(n)$	$O(1)$
ByteSampler (ours)	H	Yes*	$O(1)$	$O(1)$

Prompt Boundary Problem. Issues surrounding tokenization have been extensively documented in prior work. The prompt boundary problem was presented for maximum prefix encoding in Phan et al. (2024) and for BPE tokenizers in Vieira et al. (2024) and Ribeiro (2023). Many methods have been proposed to address the prompt boundary issue. One line of heuristic techniques, including token healing (Ribeiro, 2023) and its generalizations (Dagan et al., 2024; Athiwaratkun et al., 2024) perform “backtracking” by (i) removing one or more of the most recent tokens, followed by (ii) sampling a continuation of the partial prompt using the language model, constraining the newly generated tokens to match the remaining prompt.

Exact methods, which preserve the sampling distribution of the original language model, have also been proposed. Vieira et al. (2024) gave an exact method which requires exponential time as well as an approximate solution leveraging beam search. Turaga (2025) proposed a method that combines backtracking with the exponential time method of Vieira et al. (2024), adding a “back tokenization” step that significantly reduces the number of necessary calls to the language model, but still requires exponential overhead. Additionally, Phan et al. (2024; 2025) proposed an exact method which requires only linear time.

Exactness. The goal when solving the PBP is to sample text from a LM conditioned on a byte prefix. Given a LM over tokens, this conditioning can be expressed as

$$\begin{aligned} &P(\text{completion} \mid \text{prompt}) \\ &= \frac{P(\text{prompt} \# \text{completion} \sqsubseteq \text{decode}(t_1, \dots, t_n))}{P(\text{prompt} \sqsubseteq \text{decode}(t_1, \dots, t_n))}, \end{aligned} \quad (2)$$

where $t_1, \dots, t_n \sim \text{LM}$ and $\sqsubseteq$ denotes the prefix relation. This is the probability that a generation $t_1, \dots, t_n$ under the LM starts with $\text{prompt} \# \text{completion}$, given that it starts with prompt . Crucially, as we saw earlier, this is not the same distribution as the one obtained by simply tokenizing the prompt, then sampling a completion as in Eq. (1), precisely due to the PBP.

Following Phan et al. (2024), we consider a method “exact” if its outputs match Eq. (2) up to the probability mass the LM assigns to invalid token sequences. Although LMs are trained not to produce such sequences, they may still do so erroneously. We measure the probability mass placed on invalid token sequences in Section E.4 and discuss this assumption and implications for utility when it does not hold in more detail in Section D.

3. Method

In this section, we describe the core mechanism behind ByteSampler. In Section 3.1 we define the *Valid Covering Tree* (VCT), an object containing all of the token sequences that cover a prompt. In Section 3.2, we show how the VCT can then be used to perform standard language modeling tasks: computing prefix probabilities, sampling completions while avoiding the prompt boundary problem, and computing next-byte distributions. In Section 3.3, we show that the VCT has bounded size, leading to low inference costs. Finally, Sections 3.4 and 3.5 shows how to update the VCT incrementally as new bytes are generated.

3.1. Valid Covering Trees

Definition 3.1 (Valid Covering Tree). Let P be a byte string and let decode be the tokenizer’s decoding function.⁴ The *Valid Covering Tree* of P , denoted $\text{VCT}(P)$, is the tree of all finite token sequences $T = [t_1, \dots, t_n]$ such that:

1. P is a prefix of $\text{decode}(T)$,
2. $\text{decode}(t_1, \dots, t_{n-1})$ is a prefix of P , and
3. T is a valid token sequence.

Condition 1 ensures that the token sequence covers the prompt, Condition 2 ensures that it *minimally* covers the

⁴In our convention, every sequence begins with BOS , and $\text{decode}(\text{BOS})$ is the empty string.

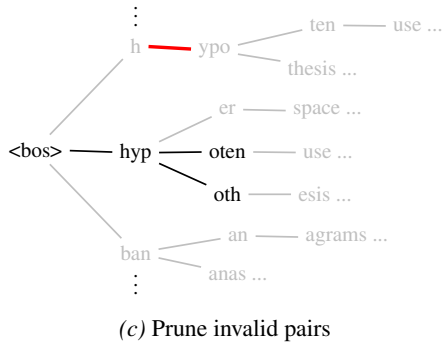
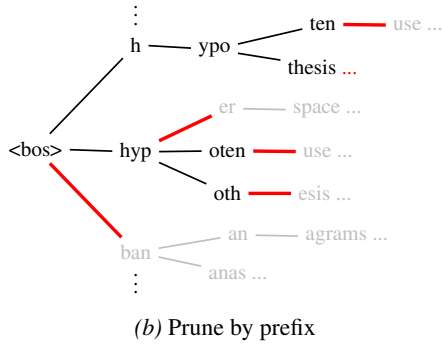
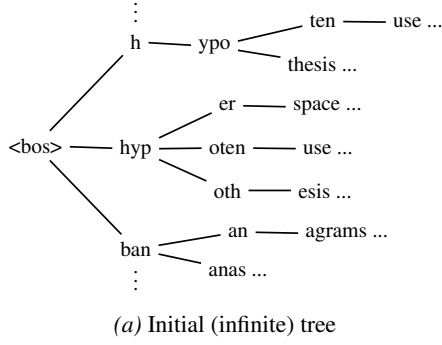


Figure 2. Construction of the Valid Covering Tree for string prefix “hypot”: (a) starting with the infinite tree of all possible token sequences (many edges not shown), we prune branches that (b) do not match the given prefix or begin after the prefix ends or (c) contain invalid contiguous pairs of tokens. More example trees are shown in Section G.

prompt (i.e., only the final token straddles the end of the prompt), and Condition 3 enforces token sequence validity (i.e., that it is in the output space of encode). As we will later see in Proposition 3.4, it is sufficient to establish token sequence validity via token pairwise validity.

3.2. Language Modeling With the Valid Covering Tree

The VCT T for a given a byte-string S can be used to efficiently perform various byte-level language modeling

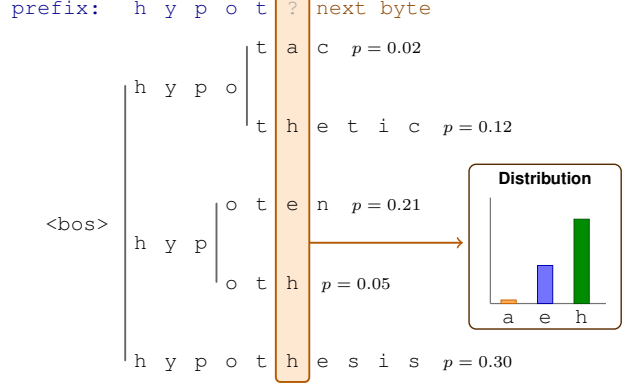


Figure 3. Computing the next-byte distribution after prefix `hypot` using its Valid Covering Tree. Leaves are grouped by their next byte after the prefix. Their probabilities are summed to obtain the byte-level distribution.

tasks. We use “ByteSampler” to refer to this collection of routines.

1. To **compute the probability of S (as a prefix)** under the LM, we sum the cumulative probabilities the LM assigns to the sequences represented by all leaves of T .
2. To **sample a completion of S while avoiding the PBP**, we compute the probability (as above) of every leaf in T and sample one of them accordingly. We are then free to continue sampling a continuation from that leaf using normal token-level sampling because sampled tokens induce token boundaries selected by the model. This one-time operation can be used to solve the PBP without paying the cost of byte-level sampling.
3. To **compute the next byte distribution following S** , we group the leaves of T by their corresponding next byte and sum the probabilities of the leaves in each group, as illustrated in Fig. 3. By repeatedly sampling from this distribution, we can generate text one byte at a time. Naturally, this will generate text more slowly than sampling at the token level. We quantify this overhead in Section 4.2.

3.3. Properties of the Valid Covering Tree

The tree depicted in Fig. 2b corresponds to the *cover* described in Vieira et al. (2024), which notes that it will generally have exponential size in the length of the prefix. In contrast, the VCT is a much smaller subtree with a useful trunk-and-branches structure.

Proposition 3.2. *Let P be a byte string and let $T = \text{VCT}(P)$. Then there exists an integer $L \leq L_0$, where L_0 depends only on the tokenizer, and a decomposition $P = P_{\text{trunk}} \# P_{\text{suffix}}$ with $|P_{\text{suffix}}| = L$, such that:*

1. every path in T shares the prefix $\text{encode}(P_{\text{trunk}})$, and
2. the number of edges of T outside this shared trunk is bounded by a constant depending only on the tokenizer.

The proof is given in Section D.1. Intuitively, Berglund & van der Merwe (2023) show that BPE tokenization needs only a constant amount of lookahead to determine the next output token. Once we are more than L_0 bytes away from the end of the prompt, the tokenization is therefore forced, so the VCT decomposes into a deterministic trunk plus a bounded branching suffix.

This compactness has an immediate algorithmic consequence. If the shared trunk has length $|\text{encode}(P_{\text{trunk}})|$, then the entire VCT can be scored using at most

$$|\text{encode}(P_{\text{trunk}})| + L \leq |\text{encode}(P)| + L_0$$

LM forward passes, because each branching prefix can score all of its possible final-token continuations in one pass. Thus solving the prompt boundary problem adds only constant overhead in token evaluations relative to standard token-level sampling.

The VCT therefore has several properties which will prove useful:

1. **Correctness:** By Definition 3.1, the tree represents exactly the set of valid token sequences whose decodings have the prompt as a prefix. (See also Sections 3.4 and D.)
2. **Compactness:** By Proposition 3.2, the tree is composed of a deterministic trunk of tokens plus a finite branching suffix whose size is bounded by a tokenizer-dependent constant.

Additional implementation details and optimizations are presented in Section C.

3.4. Pairwise Validation

We begin by formalizing the notion of validity for token sequences, following Phan et al. (2024).

Definition 3.3 (Valid token sequence). For a token sequence $T = [t_1, t_2, \dots, t_n] \in V^n$, we say that T is *valid* if $\text{encode}(\text{decode}(T)) = T$.

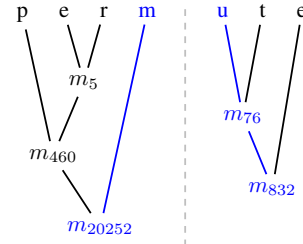
The correctness of the pairwise pruning depends on the following proposition regarding validity under BPE tokenization. (We give a proof for completeness in Section D.2.)

Proposition 3.4 (Pairwise validity, van Antwerpen & Neubeck (2025)). Let $(\text{encode}, \text{decode})$ denote a BPE encoder and decoder pair corresponding to some merge list

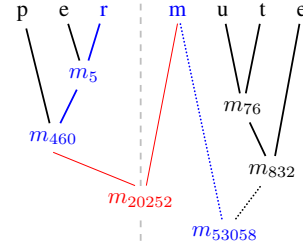
M and vocabulary V . Let $T = [t_1, t_2, \dots, t_n] \in V^n$. Then T is valid if and only if $[t_i, t_{i+1}]$ is valid for all $i \in \{1, \dots, n-1\}$.

To see that this proposition is true, consider two valid token sequences $T_1 = \text{encode}(S_1)$ and $T_2 = \text{encode}(S_2)$ and note that the concatenation $T_1 + T_2$ is valid if and only if there is no merge applied that crosses the boundary between S_1 and S_2 while tokenizing $S_1 + S_2$. We depict an example of both cases using OpenAI’s cl100k tokenizer (OpenAI, 2023) in Fig. 4.⁵

If there is no such merge, then the two strings are effectively tokenized separately, so $\text{encode}(S_1 + S_2) = T_1 + T_2$ and thus $T_1 + T_2$ is valid. On the other hand, if there is such a merge, then $\text{encode}(S_1 + S_2)$ must feature a token crossing the boundary (since no merge can be undone), which means $T_1 + T_2$ cannot be valid since it has no such token.



(a) Valid pair: no merge crossing boundary



(b) Invalid pair: merge m_{20252} crosses boundary

Figure 4. Example of valid and invalid token pairs. We show the initial string’s bytes and the merges $m_t \in M$ that are applied to the string (in order of t) to tokenize the string. In the invalid case, merge m_{53058} cannot occur because a conflicting merge m_{20252} was applied earlier. The key observation is that we only need to consider the trajectory at the boundary (in blue) to decide if the pair is valid.

This implies a fast method to test whether a pair of tokens is valid: we inspect the merge trajectory along the boundary between the tokens and check if any conflicting merges would be applied. The worst case merge tree depth is fixed by the tokenizer, so this check can be done in constant time.⁶

⁵It’s worth noting that the analogs of Proposition 3.4 do *not* hold for either Unigram (Kudo & Richardson, 2018) or Wordpiece (Schuster & Nakajima, 2012) tokenizers.

⁶We generally expect the depth of the merge trees to scale with

Table 2. Inference cost of various exact solutions to the prompt boundary problem. Our method has 65% less overhead than the next best method. Overhead vs. BPE measures the average additional tokens of inference required by the method, compared to plain BPE. Importantly, the overhead is paid for each byte when sampling at the byte level, making low overhead crucial for efficient sampling.

Method	Inference Tokens	Overhead vs. BPE
No mitigation (plain BPE)	23.51	0
Prefix Covering (Vieira et al., 2024)	2.12×10^{30}	$+2.12 \times 10^{30}$
Phan et al. (2025)	72.99	+49.47
Phan et al. (2025) with prefix caching	25.61	+2.09
ByteSampler (ours) ⁷	24.24	+0.72

3.5. Incrementally Updating the Valid Covering Tree

We can use the Valid Covering Tree as the basis for a streaming algorithm by incrementally updating it to reflect newly sampled bytes. Given a stream of input bytes, we will use Algorithm 2 to update “branches” of the Valid Covering Tree, while writing the fully determined “trunk” of tokens to an output stream.

Algorithm 2: Streaming BPE tokenization maintaining a tree matching Fig. 2c

Input: Branching tree T , new byte b
Output: stream of fully determined tokens
for every node N that ends before b begins **do**
 add all *valid* next tokens as children of N ;
 // See Fig. 2c
end
Prune branches that do not match b ;
// See Fig. 2b
while the root of T has only one child **do**
 Add the root token to the output stream and
 make its only child the new root;
end

This routine is efficient because the tree T always has a bounded size (as shown in Section D.1). This means that both expanding nodes to extend the tree and pruning nodes that do not match the new byte can be done in constant time. For concrete results, see Section 4.1, where we show that the tree has only 0.72 extra non-leaf nodes on average.

3.6. Other Tokenizer Features

Modern tokenizers are often complex pipelines consisting of many stages and additional features. To ensure the VCTs we generate are valid, we must carefully consider each feature’s impact on the tree of possible valid token sequences. We consider pretokenization in Section C.3, special tokens in Section C.4, conversion of merge lists to normal form in

the logarithm of the vocabulary size V , although we ignore scaling with respect to the tokenizer’s parameters for brevity.

Section C.5, and miscellaneous other tokenizer features in Section C.6.

4. Experiments

In our experiments, we apply ByteSampler at inference time to off-the-shelf language models. In Section 4.1 we show that our method has less computational overhead compared to other exact methods. Next, in Section 4.2, we show that exact methods perform better than heuristics in character-level language modeling. Finally, we present several applications of our method to enable higher-level functions such as ensembling (Section 4.3) and proxy-tuning (Section 4.4) models with mismatched tokenizers.

4.1. Efficiency

As discussed in Section 2, there are several existing methods which are also “exact.” Although each technically corresponds to a different sampling distribution, we do not expect there to be any significant differences between them in practice. Therefore, the main distinguishing factor to consider is the method’s computational cost.

Solving the PBP. To estimate the overhead of solving the PBP in a realistic setting, we sample a random 100 character substring from the OLMO2 pretraining corpus (OLMo et al., 2024) and estimate how many inference tokens (according to the OLMO2 tokenizer) each method needs to solve the PBP.⁹ Note that the substring is sampled uniformly, so it is about 80% likely to end in the middle of a word. We report the average inference cost in tokens, averaged over 10,000 samples, for several methods in Table 2.

Byte-level sampling. In Section E.1, we compare the performance (speed) of ByteSampler with the method of Phan et al. (2025). We find that we use $2.7\times$ less wall-clock time and use $3.3\times$ fewer inference tokens to sample the same

⁹For ByteSampler, calculating the probability of the string as a byte prefix has the same cost.

⁷We believe Back Tokenization (Turaga, 2025) should match our method when it comes to required inference tokens. However, its worst-case exponential *overhead* limits its practicality.

Table 3. Language modeling loss of OLMo2-1B on English text using various methods. We compare three settings: (i) the original token-level cross-entropy loss when predicting the next token; (ii) the character-level loss when predicting the next character by directly tokenizing the prompt and calculating the next character distribution; and (iii) the character-level loss obtained using ByteSampler to predict the next character. The higher loss per unit for token-level prediction is to be expected, as tokens are harder to predict than bytes. Once the loss is normalized to bits per character, our method and the original model achieve similar results, which demonstrates that our method does not degrade language modeling quality.

Prediction unit	Method	Loss per unit	Bits per character ⁸
Token	Plain BPE	2.67	0.85
Character	No mitigation (plain BPE)	4.81	6.94
Character	ByteSampler (ours)	0.60	0.87

number of bytes. In Section E.2, we compare with the Beam Summing algorithm of (Vieira et al., 2024). We find that, even with beam size $K = 1$, we sample bytes at roughly the same speed, but process prompts $8.6\times$ faster. Compared to standard token-level sampling (Section E.3), PBP correction has negligible overhead, and full byte-level sampling operates near the intrinsic bytes-per-token cost.

4.2. Character-Level Language Modeling

In this section, we will focus on converting off-the-shelf language models into character-level language models.¹⁰ We then evaluate the character-level prediction performance using the standard cross-entropy loss as well as next-character prediction accuracy in two languages: English in Section 4.2.1 and Chinese in Section 4.2.2.

4.2.1. OLMo2 FOR ENGLISH TEXT

In this setting, we sample a document randomly from the OLMo2 pretraining corpus (OLMo et al., 2024) and choose a random prefix of length at most 1000 characters. We then compute the next-character distribution according to OLMo2-1B (Team, 2025b) using various methods. To allow comparison with the original token-based model, we also truncate the prefix to the nearest token boundary and perform next-token prediction with the original model. We can compare the character-level and token-level losses via bits per character (Mielke, 2019), which normalizes the loss to account for the fact that tokens are more difficult to predict due to their greater information content. We report the average loss of the predictions over 100,000 such documents in Table 3.

We can clearly see the effect of the prompt boundary problem: naively predicting the next character by directly applying the tokenizer to an arbitrary string prefix as in Eq. (1) leads to poor performance (“no mitigation” in Table 3). In contrast, ByteSampler nearly matches the performance of

the original token-based model (“plain BPE”) in bits per character, as expected for exact methods.

For backtracking methods, it is not easy to compute the probability of any particular next character. This prevents us from calculating the cross-entropy loss as in Table 3. For our experiments, we compare to the Token Alignment method of Athiwaratkun et al. (2024), which is the most advanced of the proposed backtracking methods and also includes token healing as a special case. We use it to directly predict the next character by sampling greedily and report the average accuracy over 100,000 samples in Table 4.

Interestingly, we find that too much backtracking hurts the performance of the Token Alignment method. We believe this is because the sampling step often segments the remainder of the prompt in a non-standard way, which may harm the performance of the model.

4.2.2. QWEN3 FOR CHINESE TEXT

Since Chinese writing does not use whitespace, ending the prompt with a complete word does not generally provide a reliable token boundary. This makes it more difficult to heuristically avoid the PBP. Similar to Section 4.2.1, we sample a random prefix of length at most 500 characters of a random document from the Chinese subset of the MADLAD-400 dataset (Kudugunta et al., 2023). We then compute the next-character distribution according to QWEN3-1.7B-BASE (Team, 2025c) using various methods and report the average cross-entropy loss over 100,000 documents in Table 5.

Once again, the naive method fails while our method achieves similar normalized loss to the original token-level model. We also report next character prediction accuracy to allow comparison with backtracking methods. Note that Chinese has much higher entropy at the character level so the average accuracies are proportionally lower.

¹⁰We choose character-level modeling for this section, even though our method supports byte-level predictions, because some related methods can only operate on character strings.

⁸For token level prediction, calculated using a conversion rate of 4.518 characters per token (average over an independent 25M character sample from the data).

Table 4. Next character prediction accuracy of OLMo2-1B on English text using various methods. We compare three settings (i) directly tokenizing the prompt and greedily sampling until the first character of the completion is determined¹¹; (ii) using backtracking with Token Alignment (of which Token Healing is a special case) to predict the next character; and (iii) using ByteSampler to predict the next character. Overhead vs. BPE measures the average additional tokens of inference required by the method, compared to (i).

Method	Next character accuracy	Overhead vs. BPE
No mitigation (plain BPE)	29.490	0
1 Token Backtracking (Token Healing)	71.634	+0.43
2 Token Backtracking (Token Alignment)	76.281	+0.53
4 Token Backtracking (Token Alignment)	75.407	+1.08
ByteSampler (ours)	81.560	+1.72

Table 5. Language modeling loss of QWEN3-1.7B-BASE on Chinese text using various methods. We use the same settings and metrics as Table 3. Similarly to our English results, ByteSampler achieves a similar normalized language modeling loss (in bits per character) to the original model which can only perform next token prediction.

Prediction unit	Method	Loss per unit	Bits per character ¹²
Token	Plain BPE	3.43	3.50
Character	No mitigation (plain BPE)	3.79	5.47
Character	ByteSampler (ours)	2.38	3.43

4.3. Byte-Level Ensemble

Another application enabled by byte-level sampling is the ensembling of language models with different tokenizers. In general, when vocabularies between LMs are the same, their next-token probability or logit distribution can be combined via arithmetic into a single distribution, but this cannot be done directly when the vocabularies differ. Several works have proposed methods to combine LM predictions despite mismatching vocabularies (Kasai et al., 2022; Lv et al., 2024; Liu et al., 2024d; Xu et al., 2024a), but these may introduce bias into the sampling distribution. Our method makes the direct ensemble possible by converting models with BPE tokenizers into a byte-wise models, thus unifying their vocabularies.

In our experiment, we consider an ensemble of three small base language models: QWEN3-1.7B (Team, 2025c), OLMo2-1B (OLMo et al., 2024; Team, 2025b), and LLAMA3.2-1B (Team, 2024c). We combine the predictions by computing the average $p_{\text{ensemble}} = \frac{1}{n} \sum_{i=1}^n p_i$ where $p_1, \dots, p_n$ are the next-byte probability distributions for each model. We evaluate the models on a suite of seven tasks and report the results in Table 7.

4.4. Byte-Level Proxy-Tuning

In addition to additive ensembles over probabilities, the *logit*-level predictions of multiple LMs can be combined via arithmetic, with individual LMs acting as “experts” (if their predictions are combined additively) or “anti-experts” (if subtractively) (Liu et al., 2021; Li et al., 2023a; Shi et al., 2024b; Gera et al., 2023; Chuang et al., 2024; Shi et al.,

2024a). In particular, this form of ensembling can be used to achieve the *effect* of tuning a large pretrained LM without accessing model weights. To see how this can be done, note that clearly for logit vectors

$$\ell_{\text{tuned}} = \ell_{\text{base}} + (\ell_{\text{tuned}} - \ell_{\text{base}}).$$

The idea of *proxy-tuning* (Liu et al., 2024c) is to approximate the term $\ell_{\text{tuned}} - \ell_{\text{base}}$ using the difference between a pair of tuned and base proxy models $\ell_{\text{expert}} - \ell_{\text{anti-expert}}$. In our experiments, we proxy-tune a strong base model, LLAMA-3.1-8B, using OLMo2-1B-INSTRUCT and OLMo2-1B as the expert and anti-expert, respectively, which together represent a strong post-training recipe (OLMo et al., 2024; Lambert et al., 2025).

Shown in Table 8, we find that the proxy-tuned LLAMA 3.1 (Team, 2024b) model consistently outperforms the base model alone as well as the small tuned expert. This highlights a practical application of ByteSampler to “apply” post-training to base models without actually training them, thus disentangling the quality of the base model from that of the post-training recipe.

5. Conclusion

In this work, we introduced ByteSampler, an algorithm that eliminates the Prompt Boundary Problem by converting

¹¹This is necessary because, for byte-level BPE, a token might be a partial character.

¹²For token level prediction, calculated using a conversion rate of 1.415 characters per token (average over an independent 25M character sample from the data).

Table 6. **Next character prediction accuracy of QWEN3-1.7B-BASE on Chinese text using various methods.** We use the same settings and metrics as Table 4. Similar to our English language results, ByteSampler achieves the best prediction accuracy, but unlike in English, ByteSampler also requires the least overhead of all methods. This highlights that languages with multi-byte characters¹³ can behave differently than ones which typically use a single byte for each character.

Method	Next character accuracy	Overhead vs. BPE
No mitigation (plain BPE)	32.8	0
1 Token Backtracking (Token Healing)	49.2	+1.82
2 Token Backtracking (Token Alignment)	49.6	+2.98
4 Token Backtracking (Token Alignment)	49.0	+5.30
ByteSampler (ours)	52.7	+1.60

Table 7. **Byte-level ensemble results.** We report the performance (accuracy) of a byte-level ensemble of three models on downstream evals, along with the individual performance of each model. We see that the ensemble is competitive with the best individual model on each task and consistently outperforms the average performance across the three models. All 95% confidence intervals are smaller than ± 0.014 . We give more details regarding the evaluation in Section B.2.

Task	QWEN3	OLMO2	LLAMA3.2	Average	Ensemble
Arithmetic (Brown et al., 2020)	0.974	0.838	0.831	0.881	0.978
DROP (Dua et al., 2019)	0.470	0.409	0.299	0.393	0.479
Jeopardy (Tunguz, 2019b)	0.274	0.327	0.264	0.288	0.347
LAMBADA (Paperno et al., 2016)	0.727	0.628	0.510	0.622	0.755
SQuAD (Rajpurkar et al., 2016)	0.845	0.802	0.694	0.780	0.836
TriviaQA (Joshi et al., 2017)	0.389	0.535	0.443	0.456	0.526
WikidataQA (BIG-bench, 2023)	0.689	0.643	0.658	0.663	0.719

Table 8. **Proxy tuning results.** We report performance on downstream evaluations (with 95% confidence intervals) when proxy-tuning LLAMA3.1-8B using OLMO2-1B-INSTRUCT as the expert and OLMO2-1B as the anti-expert. We see that the proxy tuned model gains the instruction-following capability (AlpacaEval 2) and chain-of-thought capabilities (GSM8K, MMLU) of OLMO2-1B-INSTRUCT while also benefiting from its larger size, allowing it to surpass the expert’s individual performance. For details regarding the evaluation, see Section B.3.

Task	Metric	LLAMA3.1	OLMO2 INST.	LLAMA3.1 (Proxy Tuned)
AlpacaEval 2	LC winrate	0.88 $\pm$ 0.18	33.5 $\pm$ 0.9	33.5 $\pm$ 0.9
GSM8K	5 ICE, CoT, EM	55.3 $\pm$ 2.6	51.9 $\pm$ 2.7	76.6 $\pm$ 2.2
MMLU	0 ICE, CoT, MC	27.8 $\pm$ 0.7	35.2 $\pm$ 0.8	59.5 $\pm$ 0.8

any BPE tokenizer-based language model into a byte-level model while preserving its generative distribution at the text level. Interesting extensions of this method include automatic support for arbitrary pretokenizers (discussed in Section C.3), and generalization to other tokenization schemes (such as Unigram (Kudo & Richardson, 2018), Wordpiece (Schuster & Nakajima, 2012), and other variants of BPE (Provilkov et al., 2020; Chizhov et al., 2024)).

Beyond correcting sampling artifacts at the prompt-boundary—which is useful in its own right in many situations—the ability to unify vocabularies at inference time enables many forms of model composition, including ensembles of (and post-training transfer between) models with different tokenizers. Other applications of this technology include (i) byte-level knowledge distillation to transfer skills more effectively between models with different to-

kenizers, (ii) rapid post-training research leveraging the fact that a post-training recipe (represented by a pair of proxy-tuning experts) can be applied to any number of models without additional training, (iii) routing dynamically between models (Zheng et al., 2025) during generation without requiring matching tokenizers, and potentially (iv) more convenient LM-powered compression of byte streams.

In general, whenever (mismatching) tokenizers represent an obstacle or inconvenience, our method has the potential to completely bypass it at the cost of (minimally) increased inference compute. We hope that this will prove useful to LM researchers and users alike.

¹³Chinese typically uses three bytes for each character when encoded using UTF-8.

Acknowledgments

We would like to thank Hao Xu and Ke Hayase for helping us brainstorm a good example of the PBP in Chinese. JH and AL are supported by the NSF Graduate Research Fellowship Program. This work was partially funded by NSF 2113530, 2112471, 2505865, 2502281 and 2229876, and Microsoft Grant for Customer Experience Innovation.

Impact Statement

This work improves the reliability of language-model sampling by reducing tokenization-induced artifacts. These improvements may benefit users in multilingual and code-generation settings where token boundaries are misaligned with linguistic or syntactic boundaries. Our work also makes various forms of model composition that previously required matching vocabularies significantly more practical.

References

- Abdin, M., Aneja, J., Behl, H., Bubeck, S., Eldan, R., Gunasekar, S., Harrison, M., Hewett, R. J., Javaheripi, M., Kauffmann, P., et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.
- Adler, B., Agarwal, N., Aithal, A., Anh, D. H., Bhattacharya, P., Brundyn, A., Casper, J., Catanzaro, B., Clay, S., Cohen, J., et al. Nemotron-4 340b technical report. *arXiv preprint arXiv:2406.11704*, 2024.
- Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., Arora, R. K., Bai, Y., Baker, B., Bao, H., et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Ahia, O., Kumar, S., Gonen, H., Hofmann, V., Limisiewicz, T., Tsvetkov, Y., and Smith, N. A. MAGNET: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=1e3MOwHSIX>.
- Allal, L. B., Lozhkov, A., Bakouch, E., von Werra, L., and Wolf, T. SmolLM - blazingly fast and remarkably powerful. <https://huggingface.co/blog/smollm>, July 2024. Hugging Face blog — introduces the SmolLM family (135M, 360M, 1.7B) and the SmolLM-Corpus. Accessed 2025-09-24.
- Allal, L. B., Lozhkov, A., Bakouch, E., Blázquez, G. M., Penedo, G., Tunstall, L., Marafioti, A., Kydlíček, H., Lajarán, A. P., Srivastav, V., et al. SmolLM2: When smol goes big—data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
- Athiwaratkun, B., Wang, S., Shang, M., Tian, Y., Wang, Z., Gonugondla, S. K., Gouda, S. K., Kwiatkowski, R., Nallapati, R., Bhatia, P., and Xiang, B. Token alignment via character matching for subword completion. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 15725–15738, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.929. URL <https://aclanthology.org/2024.findings-acl.929>.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Bakouch, E., Ben Allal, L., Lozhkov, A., Tazi, N., Tunstall, L., Patiño, C. M., Beeching, E., Roucher, A., Reedi, A. J., Gallouédec, Q., Rasul, K., Habib, N., Fourrier, C., Kydlíček, H., Penedo, G., Larcher, H., Morlon, M., Srivastav, V., Lochner, J., Nguyen, X.-S., Raffel, C., von Werra, L., and Wolf, T. SmolLM3: smol, multilingual, long-context reasoner. <https://huggingface.co/blog/smollm3>, July 2025. Hugging Face blog post announcing SmolLM3 (3B, long-context up to 128k). Accessed 2025-09-24.
- Bavarian, M., Jun, H., Tezak, N., Schulman, J., McLeavey, C., Tworek, J., and Chen, M. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.
- Berglund, M. and van der Merwe, B. Formalizing bpe tokenization. In *13th International Workshop on Non-Classical Models of Automata and Applications, NCMA 2023, 18-19 September, 2023, Famagusta, Cyprus*, pp. 16–27. Open Publishing Association, 2023.
- Beurer-Kellner, L., Fischer, M., and Vechev, M. Guiding llms the right way: fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 3658–3673, 2024.
- Bi, X., Chen, D., Chen, G., Chen, S., Dai, D., Deng, C., Ding, H., Dong, K., Du, Q., Fu, Z., et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.
- Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O’Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.
- BIG-bench. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transac-*

- tions on Machine Learning Research, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=uyTL5Bvosj>.
- Black, S., Biderman, S., Hallahan, E., Anthony, Q., Gao, L., Golding, L., He, H., Leahy, C., McDonnell, K., Phang, J., et al. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745*, 2022.
- Blakeman, A., Basant, A., Khattar, A., Renduchintala, A., Bercovich, A., Ficek, A., Bjorlin, A., Taghibakhshi, A., Deshmukh, A. S., Mahabaleshwarkar, A. S., et al. Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models. *arXiv preprint arXiv:2504.03624*, 2025.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, pp. 1877–1901, 2020.
- Cao, K. and Rimell, L. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2104–2114, 2021.
- Chatzi, I., Corvelo Benz, N. L., Tsirtsis, E., and Gomez Rodriguez, M. Canonical autoregressive generation. *arXiv preprint arXiv:2506.06446*, 2025.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, Y., Marchisio, K., Raileanu, R., Adelani, D., Stenortorp, P., Riedel, S., and Artetxe, M. Improving language plasticity via pretraining with active forgetting. In *Advances in Neural Information Processing Systems*. NeurIPS, 2023.
- Chen, Z., Li, J., Chen, P., Li, Z., Sun, K., Luo, Y., Mao, Q., Yang, D., Sun, H., and Yu, P. S. Harnessing multiple large language models: A survey on llm ensemble. *arXiv preprint arXiv:2502.18036*, 2025.
- Chirkova, N., Kruszewski, G., Rozen, J., and Dymetman, M. Should you marginalize over possible tokenizations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–12, 2023.
- Chizhov, P., Arnett, C., Korotkova, E., and Yamshchikov, I. P. Bpe gets picky: Efficient vocabulary refinement during tokenizer training. *arXiv preprint arXiv:2409.04599*, 2024.
- Chuang, Y.-S., Xie, Y., Luo, H., Kim, Y., Glass, J. R., and He, P. Dola: Decoding by contrasting layers improves factuality in large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- Clark, J. H., Garrette, D., Turc, I., and Wieting, J. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022. doi: 10.1162/tacl_a_00448. URL <https://aclanthology.org/2022.tacl-1.5>.
- Dagan, G., Synnaeve, G., and Rozière, B. Getting the most out of your tokenizer for pre-training and domain adaptation. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Dobler, K. and De Melo, G. Focus: Effective embedding initialization for monolingual specialization of multilingual models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13440–13454, 2023.
- Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., and Gardner, M. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2368–2378, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1246. URL <https://aclanthology.org/N19-1246>.
- Gage, P. A new algorithm for data compression. *The C Users Journal archive*, 12:23–38, 1994. URL <https://api.semanticscholar.org/CorpusID:59804030>.
- Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- Gee, L., Zugarini, A., Rigutini, L., Torroni, P., et al. Fast vocabulary transfer for language model compression. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 409–416. Association for Computational Linguistics (ACL), 2022.

- Gee, L., Rigutini, L., Ernandes, M., and Zugarini, A. Multi-word tokenization for sequence compression. In Wang, M. and Zitouni, I. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 612–621, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-industry.58. URL <https://aclanthology.org/2023.emnlp-industry.58>.
- Geh, R. L., Zhang, H., Ahmed, K., Wang, B., and Broeck, G. V. d. Where is the signal in tokenization space? *arXiv preprint arXiv:2408.08541*, 2024.
- Gera, A., Friedman, R., Arviv, O., Gunasekara, C., Szajder, B., Slonim, N., and Shnarch, E. The benefits of bad advice: Autocontrastive decoding across model layers. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10406–10420, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.580. URL <https://aclanthology.org/2023.acl-long.580>.
- Groeneveld, D., Beltagy, I., Walsh, P., Bhagia, A., Kinney, R., Taffjord, O., Jha, A. H., Ivison, H., Magnusson, I., Wang, Y., et al. Olmo: Accelerating the science of language models. *arXiv preprint arXiv:2402.00838*, 2024.
- Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Del Giorno, A., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.
- Huang, Y., Feng, X., Li, B., Xiang, Y., Wang, H., Liu, T., and Qin, B. Ensemble learning for heterogeneous large language models with deep parallel collaboration. *Advances in Neural Information Processing Systems*, 37: 119838–119860, 2024.
- Jackson, J. Character prefix conditioning, 2025. URL <https://www.cursor.com/blog/cpc>.
- Javaheripi, M., Bubeck, S., Abdin, M., Aneja, J., Bubeck, S., Mendes, C. C. T., Chen, W., Del Giorno, A., Eldan, R., Gopi, S., et al. Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 1(3):3, 2023.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b. *arXiv preprint arXiv:2407.10671*, 2023.
- Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Barzilay, R. and Kan, M.-Y. (eds.), *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.
- Kasai, J., Sakaguchi, K., Le Bras, R., Peng, H., Lu, X., Radev, D., Choi, Y., and Smith, N. A. Twist decoding: Diverse generators guide each other. In Goldberg, Y., Kozareva, Z., and Zhang, Y. (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 4909–4923, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.326. URL <https://aclanthology.org/2022.emnlp-main.326>.
- Kudo, T. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75, 2018.
- Kudo, T. and Richardson, J. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, 2018.
- Kudugunta, S., Caswell, I., Zhang, B., Garcia, X., Xin, D., Kusupati, A., Stella, R., Bapna, A., and Firat, O. Madlad-400: A multilingual and document-level large audited dataset. *Advances in Neural Information Processing Systems*, 36:67284–67296, 2023.
- Kumar, D. and Thawani, A. BPE beyond word boundary: How NOT to use multi word expressions in neural machine translation. In Tafreshi, S., Sedoc, J., Rogers, A., Drozd, A., Rumshisky, A., and Akula, A. (eds.), *Proceedings of the Third Workshop on Insights from Negative Results in NLP*, pp. 172–179, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.insights-1.24. URL <https://aclanthology.org/2022.insights-1.24>.

- Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Taffjord, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL <https://arxiv.org/abs/2411.15124>.
- Li, X. L., Holtzman, A., Fried, D., Liang, P., Eisner, J., Hashimoto, T., Zettlemoyer, L., and Lewis, M. Contrastive decoding: Open-ended text generation as optimization. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12286–12312, Toronto, Canada, July 2023a. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.687. URL <https://aclanthology.org/2023.acl-long.687>.
- Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., and Lee, Y. T. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023b.
- Liu, A., Sap, M., Lu, X., Swayamdipta, S., Bhagavatula, C., Smith, N. A., and Choi, Y. DExperts: Decoding-time controlled text generation with experts and anti-experts. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 6691–6706, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.522. URL <https://aclanthology.org/2021.acl-long.522>.
- Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024b.
- Liu, A., Han, X., Wang, Y., Tsvetkov, Y., Choi, Y., and Smith, N. A. Tuning language models by proxy. In *First Conference on Language Modeling*, 2024c.
- Liu, A., Hayase, J., Hofmann, V., Oh, S., Smith, N. A., and Choi, Y. SuperBPE: Space travel for language models. *arXiv preprint arXiv:2503.13423*, 2025. URL <https://arxiv.org/abs/2503.13423>.
- Liu, C., Quan, X., Pan, Y., Lin, L., Wu, W., and Chen, X. Cool-fusion: Fuse large language models without training. *arXiv preprint arXiv:2407.19807*, 2024d.
- Liu, Y., Lin, P., Wang, M., and Schütze, H. Ofa: A framework of initializing unseen subword embeddings for efficient large-scale multilingual continued pretraining. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 1067–1097, 2024e.
- Lundberg, S. The art of prompt design: Prompt boundaries and token healing, 2023. URL <https://medium.com/towards-data-science/the-art-of-prompt-design-prompt-boundaries-and-token-healing-3b2448b0be38>.
- Lv, B., Tang, C., Zhang, Y., Liu, X., Yu, Y., and Luo, P. Specfuse: Ensembling large language models via next-segment prediction. *arXiv preprint arXiv:2412.07380*, 2024.
- Marchisio, K., Lewis, P., Chen, Y., and Artetxe, M. Mini-model adaptation: Efficiently extending pretrained models to new languages via aligned shallow training. In *The 61st Annual Meeting Of The Association For Computational Linguistics*, 2023.
- Mavromatis, C., Karypis, P., and Karypis, G. Pack of llms: Model fusion at test-time via perplexity optimization. In *First Conference on Language Modeling*, 2024.
- Meta. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Mielke, S. J. Can you compare perplexity across different segmentations?, Apr 2019. URL <https://sjmielke.com/comparing-perplexities.htm>.
- Minixhofer, B., Paischer, F., and Rekabsaz, N. Wechsel: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 3992–4006, 2022.
- Minixhofer, B., Ponti, E., and Vulić, I. Zero-shot tokenizer transfer. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Minixhofer, B., Vulić, I., and Ponti, E. M. Universal cross-tokenizer distillation via approximate likelihood matching. *arXiv preprint arXiv:2503.20083*, 2025.
- Nawrot, P., Chorowski, J., Lancucki, A., and Ponti, E. M. Efficient transformers with dynamic token pooling. In

- Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6403–6417, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.353. URL <https://aclanthology.org/2023.acl-long.353>.
- Oh, B.-D. and Schuler, W. Leading whitespaces of language models’ subword vocabulary pose a confound for calculating word probabilities. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3464–3472, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.202. URL <https://aclanthology.org/2024.emnlp-main.202>.
- OLMo, T., Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., Arora, S., Bhagia, A., Gu, Y., Huang, S., Jordan, M., Lambert, N., Schwenk, D., Tafjord, O., Anderson, T., Atkinson, D., Brahman, F., Clark, C., Dasigi, P., Dziri, N., Guerquin, M., Ivison, H., Koh, P. W., Liu, J., Malik, S., Merrill, W., Miranda, L. J. V., Morrison, J., Murray, T., Nam, C., Pyatkin, V., Rangapur, A., Schmitz, M., Skjongsberg, S., Wadden, D., Wilhelm, C., Wilson, M., Zettlemoyer, L., Farhadi, A., Smith, N. A., and Hajishirzi, H. 2 olmo 2 furious, 2024. URL <https://arxiv.org/abs/2501.00656>.
- OpenAI. Openai platform documentation, 2023. URL <https://platform.openai.com/docs>. Accessed: 2025/05/10.
- Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. Byte latent transformer: Patches scale better than tokens, 2024. URL <https://arxiv.org/abs/2412.09871>.
- Paperno, D., Kruszewski, G., Lazaridou, A., Pham, N. Q., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Erk, K. and Smith, N. A. (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1144. URL <https://aclanthology.org/P16-1144>.
- Phan, B., Havasi, M., Muckley, M., and Ullrich, K. Understanding and mitigating tokenization bias in language models, 2024. URL <https://arxiv.org/abs/2406.16829>.
- Phan, B., Amos, B., Gat, I., Havasi, M., Muckley, M. J., and Ullrich, K. Exact byte-level probabilities from tokenized language models for fim-tasks and model ensembles. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Pimentel, T. and Meister, C. How to compute the probability of a word. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18358–18375, 2024.
- Provilkov, I., Emelianenko, D., and Voita, E. Bpe-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020.
- Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. SQuAD: 100,000+ questions for machine comprehension of text. In Su, J., Duh, K., and Carreras, X. (eds.), *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1264. URL <https://aclanthology.org/D16-1264>.
- Ribeiro, M. T. A guidance language for controlling large language models, 2023. URL <https://github.com/guidance-ai/guidance?tab=readme-ov-file#text-not-tokens>.
- S., R. 4d masks support in transformers, 2024. URL <https://huggingface.co/blog/poedator/4d-masks>.
- Schuster, M. and Nakajima, K. Japanese and korean voice search. In *2012 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 5149–5152. IEEE, 2012.
- Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. In Erk, K. and Smith, N. A. (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL <https://aclanthology.org/P16-1162>.
- Shi, R., Chen, Y., Hu, Y., Liu, A., Hajishirzi, H., Smith, N. A., and Du, S. S. Decoding-time language model alignment with multiple objectives. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a.
- Shi, W., Han, X., Lewis, M., Tsvetkov, Y., Zettlemoyer, L., and Yih, W.-t. Trusting your evidence: Hallucinate less

- with context-aware decoding. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pp. 783–791, Mexico City, Mexico, June 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-short.69. URL <https://aclanthology.org/2024.naacl-short.69>.
- Tănase, A.-V. and Pelican, E. Supratok: Cross-boundary tokenization for enhanced language model performance. *arXiv preprint arXiv:2508.11857*, 2025.
- Tay, Y., Tran, V. Q., Ruder, S., Gupta, J., Chung, H. W., Bahri, D., Qin, Z., Baumgartner, S., Yu, C., and Metzler, D. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=JtBRnrIOEFN>.
- Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M. S., Love, J., et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024a.
- Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024b.
- Team, G., Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025a.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025b.
- Team, L. The llama 3 herd of models, 2024a. URL <https://arxiv.org/abs/2407.21783>.
- Team, L. Introducing llama 3.1: Our most capable models to date, 2024b. URL <https://ai.meta.com/blog/meta-llama-3-1/>.
- Team, L. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models, 2024c. URL <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>. Accessed: 2025/05/10.
- Team, L. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, Apr 2025a. URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- Team, M. A. Mistral NeMo | Mistral AI — mistral.ai. <https://mistral.ai/news/mistral-nemo>, 2024d. [Accessed 25-09-2025].
- Team, O. Olmo release notes, 2025b. URL <https://allenai.org/olmo/release-notes#olmo-2-1b>. Accessed: 2025/05/10.
- Team, Q. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2, 2024e.
- Team, Q. Qwen3: Think deeper, act faster, 2025c. URL <https://qwenlm.github.io/blog/qwen3/>. Accessed: 2025/05/10.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Tran, K. From english to foreign languages: Transferring pre-trained language models. *arXiv preprint arXiv:2002.07306*, 2020.
- Tunguz, B. 200,000+ jeopardy! questions, 2019a. URL <https://www.kaggle.com/datasets/tunguz/200000-jeopardy-questions>.
- Tunguz, B. 200,000+ jeopardy! questions, 2019b. URL <https://www.kaggle.com/datasets/tunguz/200000-jeopardy-questions>.
- Turaga, A. Character prefix conditioning with back tokenization, 2025. URL <https://anilturaga.github.io/cpc>.
- van Antwerpen, H. and Neubeck, A. So many tokens, so little time: Introducing a faster, more flexible byte-pair tokenizer, 2025. URL <https://github.blog/ai-and-ml/llms/so-many-tokens-so-little-time-introducing-a-faster-more-flexible-byte-pair-tokenizer/>. Accessed: 2025/05/10.
- Vieira, T., LeBrun, B., Giulianelli, M., Gastaldi, J. L., DuSell, B., Terilla, J., O’Donnell, T. J., and Cotterell, R. From language models over tokens to language models over characters. *arXiv preprint arXiv:2412.03719*, 2024.
- Vieira, T., Liu, T., Pasti, C., Emara, Y., DuSell, B., LeBrun, B., Giulianelli, M., Gastaldi, J. L., O’Donnell, T. J., and

- Cotterell, R. Language models over canonical byte-pair encodings. *arXiv preprint arXiv:2506.07956*, 2025.
- Wang, J., Gangavarapu, T., Yan, J. N., and Rush, A. M. Mambabyte: Token-free selective state space model. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=X1xNsuKssb>.
- Xu, H., Liu, A., Hayase, J., Choi, Y., and Smith, N. A. Are you going to finish that? a practical study of the partial token problem, 2026. URL <https://arxiv.org/abs/2601.23223>.
- Xu, Y., Chen, J., Wu, J., and Zhang, J. Hit the sweet spot! span-level ensemble for large language models. *arXiv preprint arXiv:2409.18583*, 2024a.
- Xu, Y., Lu, J., and Zhang, J. Bridging the gap between different vocabularies for llm ensemble. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7133–7145, 2024b.
- Xu, Y., Chen, J., Wu, J., and Zhang, J. Hit the sweet spot! span-level ensemble for large language models. In *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 8314–8325, 2025.
- Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., and Raffel, C. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. doi: 10.1162/tacl_a_00461. URL <https://aclanthology.org/2022.tacl-1.17>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Young, A., Chen, B., Li, C., Huang, C., Zhang, G., Zhang, G., Wang, G., Li, H., Zhu, J., Chen, J., et al. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*, 2024.
- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=JTmO2V9Xpz>.
- Yu, Y.-C., Kuo, C. C., Ziqi, Y., Yucheng, C., and Li, Y.-S. Breaking the ceiling of the llm community by treating token generation as a classification for ensembling. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 1826–1839, 2024.
- Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., Xie, C., Wang, C., Yin, D., Zeng, H., Zhang, J., et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
- Zhang, V., Verma, S., Vaidya, N., Sawarkar, A., and Saunders, A. Nvidia ai foundation models: Build custom enterprise chatbots and co-pilots with production-ready llms, Nov 2024. URL <https://developer.nvidia.com/blog/nvidia-ai-foundation-models-build-custom-enterprise-chatbots-and-co-pilots-with-production-ready-llms/>.
- Zheng, W., Chen, Y., Zhang, W., Kundu, S., Li, Y., Liu, Z., Xing, E. P., Wang, H., and Yao, H. Citer: Collaborative inference for efficient large language model decoding with token-level routing. *arXiv preprint arXiv:2502.01976*, 2025.

A. Related Work

Byte-level language models Although our method is able to convert a model using a traditional BPE tokenizer into a byte-level model, allowing it to be used in situations where byte-level models are required, it may not enjoy the benefits of being trained natively at the byte level. Training byte-level models are an active area of research (Clark et al., 2022; Xue et al., 2022; Wang et al., 2024). However, byte-level language models may still implicitly aggregate multiple bytes into a single “patch” to help reduce the required sequence length. These patches can be segmented either statically (Tay et al., 2022; Yu et al., 2023) or dynamically (Nawrot et al., 2023; Pagnoni et al., 2024; Ahia et al., 2024), which *may* lead to issues analogous to the Prompt Boundary Problem at the patch level, depending on the architecture.

Tokenizer transfer Methods to adapt a model to use tokenizers other than the one they are trained with have been proposed. These methods may rely on interventions during training (Chen et al., 2023), continued training for a subset of the model with the new tokenizer (Marchisio et al., 2023), using self-distillation (Minixhofer et al., 2025), careful initialization of a new embedding matrix, followed by fine-tuning (Minixhofer et al., 2022; Gee et al., 2022; Tran, 2020; Liu et al., 2024e; Dobler & De Melo, 2023), or zero shot transfer using a hypernetwork (Minixhofer et al., 2024). While these methods can, in principle, be used to convert any model into a byte-level model, they will inevitably introduce some distortion into the model’s sampling distribution.

Ensembles of language models Many methods to address the mismatching vocabularies one encounters when ensembling models have been proposed. These include bridging the vocabularies using a mapping based on model features (Huang et al., 2024) or edit distance (Mavromatis et al., 2024) as well as sampling from the union (Yu et al., 2024) or intersection (Xu et al., 2024b) of multiple vocabularies. There are also several methods that sample multiple tokens of continuation from each model and then select the best one using a scoring metric (Liu et al., 2024d; Xu et al., 2025; Lv et al., 2024). For a survey of such methods, including ones that require training or additional data, see Chen et al. (2025). However, unlike our exact method, all of these methods may introduce distortion into the model’s outputs.

Word level probabilities The popular decision to include whitespace with the following word in most modern tokenizers presents a challenge when computing the next word probability (Oh & Schuler, 2024; Pimentel & Meister, 2024), which is closely related to the Prompt Boundary Problem.

Nondeterministic tokenizers Our analysis crucially relies on the determinism of BPE, however nondeterministic tokenizers such as Unigram (Kudo, 2018) and BPE dropout (Provilkov et al., 2020) are of interest to the community. Lundberg (2023) remarks that nondeterministic tokenizers may reduce the severity of the prompt boundary problem, but it cannot do so perfectly. It is possible that more advanced techniques may be able to fully correct the PBP for these tokenizers as well.

B. Experimental Details

In this appendix, we report additional experimental details.

B.1. Calculation of the Naive Method

The naive method is simple to state. We merely report the average probability that the next character sampled after the prompt will be the correct one. However, some complexity arises when considering multibyte characters, which occur occasionally in English text and essentially constantly in Chinese. A multibyte character may correspond to multiple tokens under a byte-level BPE tokenizer, which means that multiple sampling steps may be necessary to form the next character. To handle this properly, we compute the tree of all token sequences which start with the desired character (depicted in Fig. 2b) and score the log-probability of all of its leaves to determine the exact probability that the desired next character will be generated. Note that we do not perform the pairwise pruning in this step, as we describe in Fig. 2c and Section 3.4. It is not strictly necessary, since a single character can be at most four bytes under UTF-8, so the size of the tree will always be small, and omitting the pruning step presents the baseline in the best light.

B.2. Details for Ensemble Evaluations

For the ensemble evaluations we use few-shot prompting with five in-context examples for each query. We choose the few-shot examples randomly to avoid any bias and ensure that the question being tested is not among the examples. We

sample the continuation greedily and test whether the resulting text contains the correct answer.

1. **Arithmetic** contains simple arithmetic problems (Brown et al., 2020).¹⁴ We use the 2da, 2dm, and 2ds splits for addition, multiplication, and division of (up to) 2-digit numbers.
2. **DROP** contains questions about passages, potentially requiring reasoning over multiple pieces of information in the passage (Dua et al., 2019).
3. **Jeopardy** contains open-ended questions from the “Jeopardy!” quiz show (Tunguz, 2019a).
4. **LAMBADA** contains narratives without the last word, which is inferrable given the context (Paperno et al., 2016). This task requires models to attend to the full narrative instead of only the local context.
5. **SQuAD** contains passages paired with questions about the passage (Rajpurkar et al., 2016). The answer is always a span from the passage.
6. **TriviaQA** contains open-ended questions about world knowledge (Joshi et al., 2017).
7. **BIG-bench WikidataQA** require models to complete factual statements with the correct continuation (BIG-bench, 2023).

To save compute, we randomly subsample large datasets down to 5,000 examples.

B.3. Details for Proxy-Tuning Evaluations

Following Liu et al. (2024c), we use the proper instruct template for OLMO2-INSTRUCT and use a basic Question/Answer format for the base models. Unlike in the previous section, we use a more varied evaluation setup.

1. For **AlpacaEval 2**, we prompt using the instruction as the question and take the response as the answer. This is done with no chain of thought prompting or in-context examples. We use the default AlpacaEval 2 judge and report the length-controlled win-rate in our results.
2. For **GSM8k**, we use five in-context examples, which naturally cause the model to produce chains of thought. We extract the final number produced by the model and test if it exactly matches the answer (removing any commas).
3. For **MMLU**, we use no in-context examples and use the chain-of-thought prompt from Lambert et al. (2025) to elicit chains of thought resulting in a multiple-choice answer. Unlike with the other datasets, we do not truncate MMLU to 5,000 examples since its examples are distributed across various domains. We report the multiple-choice accuracy in our results.

These evaluations were intended to benefit from instruction-following capabilities and general knowledge model performance.

B.4. Compute Resources

Our experiments were conducted with a variety of computing resources, including Nvidia A40, L40S, and A100 GPUs. Our method only requires one GPU at a time and features minimal memory overhead compared to regular sampling. We estimate that the total compute required to reproduce all of our results is less than 200 L40S hours.

C. Implementation Details and Optimizations

In this appendix, we report implementation details that improve efficiency and ensure correctness.

¹⁴<https://huggingface.co/datasets/EleutherAI/arithmetic>

C.1. Inference Optimizations

To ensure that our method is practical we employ a number of optimizations. In order to quickly compute the Valid Covering Tree, we maintain a cache of token masks which are valid following a given token and a separate cache for masks specifying tokens that begin with certain common byte prefixes. Then given a node of the tree, we can quickly expand it, as described in Algorithm 2 by fetching the relevant masks from both caches and intersecting them on the GPU to find the valid children to add.

When evaluating the probabilities of the leaves of the Valid Covering Tree, we use 4D attention masks (S., 2024) to perform inference for the entire tree in a single query. Additionally, while sampling we use KV-caching to avoid redundant computation. Combining these two techniques can lead to excessive memory usage because tokens corresponding to branches that are ultimately not selected by sampling take up space in the KV cache. To address this, we implement a copying garbage collector for the KV cache which discards such tokens from the cache. Since the GC can be run one layer at a time, its total memory overhead is negligible. When using the GC, the KV cache will store exactly one set of keys and values for each token in the *current* Valid Covering Tree, reducing the memory overhead compared to naive sampling to a constant.

We also implement batching, allowing one to sample multiple sequences of bytes in parallel, which permits better utilization of GPU resources.

C.2. Byte-Level vs. Character-Level BPE

Throughout this work, we assume that BPE is carried out at the byte level. However, the alternative, performing BPE at the character level, is also a popular choice. Our method can be extended to character-level BPE merges in a natural manner. In particular, one can perform our method at the character level instead. All the analysis we provide, including the guarantees for the Valid Covering Tree in Section 3.4, continue to hold regardless of the choice of base vocabulary. The only additional logic that needs to be implemented revolves around the handling of byte fallback, which is a feature that allows the tokenizer to represent characters that were not included in the base vocabulary explicitly using their Unicode encoding. To handle this properly, we will need to “reset” the tree whenever we encounter a character encoded using byte fallback, since BPE merges do not interact with byte fallback (essentially the byte encoded character acts as a pretokenization boundary). In order to condition on an arbitrary *byte* sequence, we must consider the possibility that a partial character will be completed to form one not in the base vocabulary, necessitating the addition of a “byte fallback” branch to the Valid Covering Tree. In all other regards, the approach is the same as the one we outline in Section 3.

C.3. Handling Pretokenization

So far, we have focused on correctly handling BPE itself, while ignoring the pretokenization conventionally applied beforehand. To illustrate why this step is important, recall that pretokenization is often used to ensure that tokens cannot span multiple words and that whitespace separating words is merged with the following word and not the preceding one. In order to correctly handle all aspects of modern tokenizers, we must also perform pretokenization in an online fashion, which is challenging in its own right.

Pretokenization is typically implemented using a regular expression: beginning at the start of the text, the longest prefix matching the regular expression is found greedily. This prefix is then extracted into a pretoken and the process is repeated on the suffix. This continues until the entire string has been processed. In order to properly handle pretokenization, we must also perform this splitting online. Due to the expressivity of regular expressions, this requires maintaining a tree of possible splits, which are resolved once enough text is observed, to conclude whether the regex will match or not.

C.3.1. GENERAL SOLUTION

In principle, the implementation of this idea is straightforward. We can convert any splitting regular expression into a finite automaton, which allows us to detect matches incrementally. By performing connectivity analysis on the automata’s state graph, we can infer (i) whether there exists a suffix that could produce a regex match (which would mean that the pretokenization might not end up splitting at this point) and also (ii) whether there exists a suffix which would cause the regex to stop matching at this point (which would mean that the pretokenization might end up splitting at this point). This analysis can be precomputed for each state in the automaton, allowing these checks to be performed in constant time for each new byte.

If the verdict is ambiguous (both splitting and not splitting are possible), then we add an additional subtree to the Valid Covering Tree which assumes that the split has indeed happened. The portion to the left of the split can only be tokenized one way (since its endpoint is fixed), while the portion to the right of the split will be isomorphic to a new Valid Covering Tree for the portion of the prefix following the hypothetical split. As we continue to add new bytes, we maintain both branches of the tree, just as we would normally. Once enough bytes are added, we can determine conclusively which option was taken, allowing us to discard the subtree corresponding to the opposite possibility.

Of course, it is possible that a new position may occur where the splitting cannot be determined conclusively before the first one is resolved. This will necessitate further splitting of the tree (potentially in both subtrees). In general, this may lead to trees of exponential size, but for typical pretokenizers in use today, we can still guarantee that the tree will have finite size.

C.3.2. PRACTICAL SOLUTION

Unfortunately, the general solution we outlined in the previous section is difficult to implement in practice. First, most regular expression engines in use today support matching features that are not strictly regular, which makes the conversion of its regexes into automata impossible in the general case. While these features are not used by any pretokenizer we are aware of, this possibility has made it difficult to find routines that can perform this conversion for existing regex engines.

Our implementation therefore takes a more direct approach. We maintain a streaming pretokenization state alongside the streaming BPE state. When the current byte prefix determines a unique pretokenization, we simply continue extending this state online. When the pretokenizer admits multiple possible split decisions at the current suffix, we temporarily branch the state, continue BPE independently on each branch, and emit only the token prefix shared by all active branches. Once enough text has been observed to determine which split actually occurs, we discard the inconsistent branches and continue with the surviving one.

To implement this efficiently, we do not attempt to support arbitrary regexes in full generality. Instead, we observe that most pretokenization rules used in practice are *closed under prefix*: whenever a string matches the rule, every prefix of that match also matches. For such rules, possible split points are easy to detect online, since once the regex stops matching, no suffix can make the current prefix match again. The remaining ambiguity comes from a small number of practically important non-prefix-closed behaviors, for which we provide dedicated handlers:

- Most tokenizers have a lookahead rule which stops matching whitespace one before the last whitespace. Thus given three spaces in a row, followed by a letter, the first two spaces would be one pretoken and the last space and letter would form a second pretoken.
- Many tokenizers have a “contraction merging” rule which forces contraction suffixes such as `<'ve>` to be individual pretokens. This is tricky because `<'ve>` is considered a match but `<'v>` is not.
- Some tokenizers align digit groups from the right, so the correct split may depend on how many additional digits arrive.

These handlers cover the unstable split patterns we have observed in modern tokenizers, while the prefix-closed case handles the rest. This is enough to correctly support all pretokenizers we are aware of, including the HuggingFace ByteLevel BPE tokenizers used by major open-weight models. See Section C.7 for a list of models tested.

C.4. Handling Special Tokens

Special tokens are tokens that are assigned special meaning and are not used simply to represent text. These tokens can have a variety of uses, including marking the beginning or end of documents or separating turns in a dialog. It is easy to handle special tokens in the prompt: when we see a special token, we terminate the tree at that point (discarding potential continuations), output the ID of the special token, and then start a new tree.

To handle special tokens in the output, we consider the special token distribution on the branch of the tree that ends exactly at the end of the prompt and add those tokens as generation options with the corresponding probabilities alongside the 256 possible next bytes. When composing multiple models which have different sets of special tokens, we require a mapping to specify which tokens have the same meaning. This mapping is automatically detected for BOS and EOS tokens, but must be manually specified for others.

C.5. Converting Merge Lists to Normal Form

Throughout this work, we have assumed that the tokenizer is constructed using the BPE training algorithm, which proceeds by iteratively merging the most frequent pair in the partially tokenized training corpus. This assumption leads to merge lists that have three desirable properties: (i) every token has a unique merge that forms it, (ii) every token can be achieved as the tokenization of some string, and (iii) the merges always appear in the order they are merged. We assume that these properties are true in the analysis we present in Section 3.

However, in practice, some models use “BPE” tokenizers with merge lists that are not directly produced by the BPE training algorithm. One example of this is the tokenizer of LLAMA 3 (Team, 2024a), which appears to be constructed by extending OpenAI’s cl100k tokenizer (OpenAI, 2023) with additional merges intended to add multilingual capabilities. Because of the way this extension is done, the LLAMA 3 tokenizer does not have any of the three properties we outlined above. Despite this, inference with the tokenizer is still possible because some tokenization libraries such as HuggingFace Tokenizers¹⁵ employ a heap-based algorithm which simply applies the earliest merge available until no more merges can be applied, which permits the merges to be applied out of order.

Fortunately, it happens to be the case that every merge list can be converted into a functionally equivalent one in “normal form” which has identical behavior while also satisfying the three properties above. This is done using a two step process: (1) for each token, we run the heap-based algorithm on it as a string and track which merges are used during the tokenization process. If the resulting token sequence is *not* just the single corresponding token id, then we mark the token as “unreachable” and drop it (this ensures property (ii)). Otherwise, we check which merge was applied last and drop any other merges which form the same token since they are also unreachable (this ensures property (i)). Then (2), for every merge we check the position of the merges forming its two inputs and move it to immediately after the later of the two if it appears after the original merge (this ensures property (iii)). This procedure allows our method to be used with any tokenizer that can be specified using a merge list, even if it was not trained using BPE.

C.6. Other Tokenizer Features

C.6.1. IGNORE MERGES

Some tokenizers have a feature called `ignore_merges`, which skips BPE when a pretoken is in the vocabulary, emitting the token directly instead. This was meant to be an optimization, but some tokenizers (like the one of LLAMA 3) have tokens which are unreachable using the BPE merges. These tokens can still be reached using the `ignore_merges` feature, so we ensure that these tokens are considered for continuation at any point where the pretokenization handler thinks a split is possible.

C.6.2. NORMALIZATION

Some tokenizers also feature a normalizer, which converts multiple byte encodings of the same character into a canonical form. For these models, we invoke the normalizer on the prompt and also normalize characters dynamically as they are generated to ensure the input distribution matches the one the LM expects.

C.6.3. ADDED TOKENS

Added tokens are matched just like special tokens (using string matching before BPE or even pretokenization), but they represent plain text strings instead of “events.” To handle these correctly, we build an Aho-Corasick style automaton which can efficiently determine all of the partial added token matches. We then carefully build the tree corresponding to these matches, splitting the tree whenever there are overlapping potential matches.

C.7. Model Support

In Table 1 we sort tokenizers into two primary categories: SentencePiece BPE and HuggingFace ByteLevel BPE. These categories cover the vast majority of modern models, including (but not limited to):

1. SentencePiece BPE: Llama 1/2 (Touvron et al., 2023a;b), Mistral (Jiang et al., 2023), Yi (Young et al., 2024)

¹⁵<https://github.com/huggingface/tokenizers>

2. HuggingFace ByteLevel BPE: GPT-OSS (Agarwal et al., 2025), Llama 3/4 (Meta, 2024; Team, 2025a), DeepSeek V1/V2/V3/R1 (Bi et al., 2024; Liu et al., 2024a;b; Guo et al., 2025), Qwen 1/2/3 (Bai et al., 2023; Team, 2024e; Yang et al., 2025), GPT-NeoX/Pythia (Black et al., 2022; Biderman et al., 2023), Mistral NeMo (Team, 2024d), OLMo 1/2 (Groeneveld et al., 2024; OLMo et al., 2024), SmoLLM 1/2/3 (Allal et al., 2024; 2025; Bakouch et al., 2025), Phi 1/2/3/4 (Gunasekar et al., 2023; Li et al., 2023b; Javaheripi et al., 2023; Abdin et al., 2024), GLM 4 (Zeng et al., 2025), Nemotron H (Blakeman et al., 2025)
3. Neither: Gemma 1/2/3 (Team et al., 2024a;b; 2025a), Kimi K2 (Team et al., 2025b), Nemotron 3/4 (Zhang et al., 2024; Adler et al., 2024)

Shown in Table 1, different exact approaches support different sets of tokenizers. Beyond implementation details, this is because Turaga (2025) and Phan et al. (2025) make an assumption about tokenizer behavior (e.g. Proposition 1 in Phan et al. (2025)) that any prefix of a valid token sequence is also a valid token sequence. However, HuggingFace tokenizers almost universally use pretokenizers that do not satisfy this assumption. In these tokenizers, the correct split at the current suffix can depend on future bytes, so the corresponding VCT requires additional branches to cover all possibilities.

For example, consider the OLMo2 tokenizer, which has tokens " " (220) (one space), " " (256) (two spaces), and "0" (15) 15 (digit 0). In this tokenizer we have $\text{encode}(\text{decode}([220, 220])) = [256]$ so $[220, 220]$ is thus invalid. However, we also have $\text{encode}(\text{decode}([220, 220, 15])) = [220, 220, 15]$ so $[220, 220, 15]$ is valid!

ByteSampler correctly handles these pretokenizers by maintaining candidate pretokenization states online, branching only over unresolved split decisions, and committing any token prefix shared across all active branches (see Section C.3). ByteSampler has been tested on all of the listed HuggingFace ByteLevel BPE tokenizers using at least 1 GB of randomly sampled fragments from The Pile (Gao et al., 2020) to ensure the computed Valid Covering Trees are correct.

D. Technical Details and Proofs

D.1. Proof of Compactness of the Valid Covering Tree

We restate Proposition 3.2 for convenience.

Proposition 3.2. *Let P be a byte string and let $T = \text{VCT}(P)$. Then there exists an integer $L \leq L_0$, where L_0 depends only on the tokenizer, and a decomposition $P = P_{\text{trunk}} \# P_{\text{suffix}}$ with $|P_{\text{suffix}}| = L$, such that:*

1. *every path in T shares the prefix $\text{encode}(P_{\text{trunk}})$, and*
2. *the number of edges of T outside this shared trunk is bounded by a constant depending only on the tokenizer.*

Proof of Proposition 3.2. By Berglund & van der Merwe (2023), there exists a tokenizer-dependent constant L_0 such that the next output token of the BPE encoder can always be determined using at most L_0 bytes of lookahead. Let $L \leq L_0$ be the smallest amount of lookahead needed to determine the first token of the encoding of P whose end position lies within the final L_0 bytes of P . Write $P = P_{\text{trunk}} \# P_{\text{suffix}}$ where $|P_{\text{suffix}}| = L$.

Item 1. Consider any sequence $T \in \text{VCT}(P)$. Let $S = \text{decode}(T)$. By definition of the VCT, P is a prefix of S , and since T is valid we have $T = \text{encode}(S)$. Now every token of $\text{encode}(S)$ whose decoded span ends before the suffix P_{suffix} is determined using only bytes from P , because at least L bytes of lookahead remain within the known prefix $P \subseteq S$. Hence those tokens are the same as in the canonical encoding of P_{trunk} , namely $\text{encode}(P_{\text{trunk}})$. Thus every path in the VCT shares the same trunk.

Item 2. After removing the shared trunk, every remaining path can be written as $T_{\text{prefix}} \# [t_{\text{end}}]$, where $\text{decode}(T_{\text{prefix}})$ is a prefix of P_{suffix} . Indeed, by definition of the VCT, only the final token may extend past P , so after the trunk is removed the remaining decoded string consists of a prefix of P_{suffix} followed by one final covering token t_{end} , whose decoded span reaches the end of P_{suffix} .

For any fixed decoded prefix string, there is at most one possible valid token sequence T_{prefix} , because a valid sequence is exactly the canonical encoding of its decoded string. Thus there is at most one such prefix node for each prefix length of P_{suffix} , including the empty prefix, so the number of such nodes is at most $L + 1$. Each of these nodes can have at most $|V|$ outgoing final-token continuations, where V is the tokenizer vocabulary. Hence the number of edges outside the trunk is bounded by $(L + 1)|V|$, which depends only on the tokenizer because $L \leq L_0$. $\square$

D.2. Proof of Proposition 3.4

First we will prove the following lemma.

Lemma D.1 (Boundary crossing merges). *Let S_1 and S_2 be strings and let $T_1 = \text{encode}(S_1)$ and $T_2 = \text{encode}(S_2)$ be their respective token sequences. Then $T_1 \# T_2$ is valid if and only if there is no merge applied that crosses the boundary between S_1 and S_2 while tokenizing $S_1 \# S_2$.*

Proof. Let $M_1 = m_1^{(1)}, \dots, m_1^{(n_1)}$ and $M_2 = m_2^{(1)}, \dots, m_2^{(n_2)}$ be the sequence of merges applied by BPE during $\text{encode}(S_1)$ and $\text{encode}(S_2)$ respectively. Every merge is represented by a tuple (t, i) where t is the index of the merge in the merge list and i is the position in the token sequence where the merge is applied and let merges be ordered lexicographically. Let $M_{12} = m_{12}^{(1)}, \dots, m_{12}^{(n_1+n_2)}$ be the sorted union of merges M_1 and merges from M_2 shifted to align with the S_2 portion of $S_1 \# S_2$ (so each $(t, i) \in M_2$ becomes $(t, i + |S_1|)$ in M_{12}).

Let the merge list $M = m^{(1)}, \dots, m^{(n)}$ be called *invalid* if there exists a merge m' which satisfies either

1. $m^{(i)} < m' < m^{(i+1)}$ and m' matches the partially merged token sequence after $m^{(i)}$,
2. $m' < m^{(1)}$ and m' matches the initial token sequence, or
3. $m^{(n)} < m'$ and m' matches the final token sequence.

If no such merge exists, the merge list is *valid*. If a merge list is valid, then it represents exactly the sequence of merges chosen by BPE, since the BPE algorithm takes the lexicographically smallest available merge at every step. Therefore, by definition M_1 and M_2 are both valid merge lists.

Now, if M_{12} is invalid, it must be due to a merge m'_{12} that was not available while tokenizing S_1 or S_2 , otherwise that merge would make M_1 or M_2 invalid. The only such merges are those that cross the boundary between S_1 and S_2 in $S_1 \# S_2$. The only such merges are those that cross the boundary between S_1 and S_2 in $S_1 \# S_2$.

If M_{12} is valid then we will have $\text{encode}(S_1 \# S_2) = T_1 \# T_2$ and so $T_1 \# T_2$ is valid.

On the other hand, if M_{12} is invalid then BPE will apply the invalidating merge m'_{12} which must cross the boundary. Then, since $T_1 \# T_2$ has no token that crosses the boundary it cannot be valid. $\square$

Now, recall the statement of Proposition 3.4,

Proposition 3.4 (Pairwise validity, van Antwerpen & Neubeck (2025)). *Let $(\text{encode}, \text{decode})$ denote a BPE encoder and decoder pair corresponding to some merge list M and vocabulary V . Let $T = [t_1, t_2, \dots, t_n] \in V^n$. Then T is valid if and only if $[t_i, t_{i+1}]$ is valid for all $i \in \{1, \dots, n-1\}$.*

Proof. Let $T = [t_1, \dots, t_n]$ and write $S_i = \text{decode}(t_i)$ for the string decoded by token t_i .

First assume $[t_i, t_{i+1}]$ is valid for all $i \in \{1, \dots, n-1\}$. We prove by induction on i that every prefix $[t_1, \dots, t_i]$ is valid. The base case $i = 1$ is immediate. For the inductive step, suppose $[t_1, \dots, t_i]$ is valid and consider appending t_{i+1} . Since $[t_i, t_{i+1}]$ is valid, Lemma D.1 implies that no merge crosses the boundary between S_i and S_{i+1} when encoding $S_i \# S_{i+1}$. Because a BPE merge combines adjacent symbols, any merge crossing the boundary between $\text{decode}([t_1, \dots, t_i])$ and S_{i+1} would in particular induce a merge crossing the boundary between S_i and S_{i+1} . Therefore no merge crosses that boundary, and Lemma D.1 shows that $[t_1, \dots, t_i, t_{i+1}]$ is valid.

Conversely, suppose $[t_i, t_{i+1}]$ is invalid for some i . By Lemma D.1, there exists a merge crossing the boundary between S_i and S_{i+1} when encoding $S_i \# S_{i+1}$. Let m' be the earliest such boundary-crossing merge. By definition of m' , no earlier merge crosses that boundary, so all earlier merges act entirely within the left or right side and cannot eliminate the adjacency required for m' . Hence the same merge m' is still available when encoding the full string $\text{decode}(T) = S_1 \# \dots \# S_n$. Therefore a merge crosses the boundary between $\text{decode}([t_1, \dots, t_i])$ and $\text{decode}([t_{i+1}, \dots, t_n])$, so Lemma D.1 implies that T is not valid. $\square$

D.3. Differences in Exact Methods

In this work, we consider a method exact if it samples according to the distribution in Eq. (2) modulo the probability mass placed on invalid sequences, which we defined in Section 3.4. Here we describe exactly how these methods differ in their handling of invalid sequences. The method of [Turaga \(2025\)](#) and our method condition on a valid covering of the prompt. This corresponds to the distribution

$$P\left(t_1, \dots, t_n \mid \begin{array}{l} \text{prompt} \sqsubseteq \text{decode}(t_1, \dots, t_n), [t_1, \dots, t_k] \text{ is valid} \\ \text{where } k = \min\{i \mid \text{prompt} \sqsubseteq \text{decode}(t_1, \dots, t_i)\} \end{array}\right). \quad (3)$$

While difficult to notate, this simply means that the portion of the sequence overlapping the prompt is required to be valid. This is roughly similar to common practice described in Eq. (1) of directly tokenizing the prompt and sampling a continuation while avoiding the PBP. Meanwhile, [Phan et al. \(2024\)](#) consider a relaxation of the above, which does not require the last pair to be valid. This corresponds to

$$P\left(t_1, \dots, t_n \mid \begin{array}{l} \text{prompt} \sqsubseteq \text{decode}(t_1, \dots, t_n), [t_1, \dots, t_{k-1}] \text{ is valid} \\ \text{where } k = \min\{i \mid \text{prompt} \sqsubseteq \text{decode}(t_1, \dots, t_i)\} \end{array}\right). \quad (4)$$

The less strict conditioning explains why this method has greater overhead, as seen in Section 4.1.

It may seem desirable to sample from the distribution

$$P(t_1, \dots, t_n \mid \text{prompt} \sqsubseteq \text{decode}(t_1, \dots, t_n), [t_1, \dots, t_k] \text{ is valid}), \quad (5)$$

where the entire sequence is required to be valid. However, it is not clear how to efficiently sample from this distribution. [Vieira et al. \(2025\)](#) highlights this difficulty and proposes several alternative approaches, including approximations of Eq. (5) and architectural modifications that make it easier to sample from Eq. (5).

When applying ByteSampler iteratively, the validity of the sequence is enforced continuously. Since this is done locally, the resulting distribution corresponds to the “locally canonicalized approximation” of Eq. (5) described in [Vieira et al. \(2025\)](#); [Chatzi et al. \(2025\)](#) additionally conditioned on a `prompt`.

D.4. Significance of Invalid Segmentations

For the most part, we have ignored the contribution of invalid sequences to the language model’s distribution. This is done out of necessity, since the number of invalid sequences scales exponentially with the prompt length ([Vieira et al., 2024](#)). However, it is worth considering whether these segmentations could contribute meaningfully to the model’s capabilities.

This is closely related to the concept of *marginalization* ([Cao & Rimell, 2021](#)): the idea that calculating the probability of generating a string with a language model requires summing over all segmentations of the string, (including invalid ones). Of note, [Chirkova et al. \(2023\)](#) found that $P([t_1, \dots, t_n] \text{ is not valid})$ makes up a negligible fraction of the language model’s distribution, however later works ([Geh et al., 2024](#); [Vieira et al., 2025](#)) came to the opposite conclusion.

D.5. Proofs of Exactness

First we show that the prefix probability calculation is exact.

Proposition D.2 (ByteSampler prefix probability exactness). *Given a byte-string P with VCT T . Let $l_1, \dots, l_\ell$ be the leaves of T and let $p_i^{(1)}, \dots, p_i^{(m_i)}$ denote the path from the root of T to l_i , then*

$$P(P \sqsubseteq \text{decode}(t_1, \dots, t_n)) \cong \sum_{i=1}^{\ell} P(p_i^{(1)}, \dots, p_i^{(m_i)})$$

where $\cong$ denotes equivalence up to the probability mass placed on invalid token sequences.

Proof. We expand the desired probability in terms of the prefix cover of [Vieira et al. \(2024\)](#) and then remove the invalid

sequences

$$\begin{aligned}
 & P(P \sqsubseteq \text{decode}(t_1, \dots, t_n)) \\
 &= \sum_{n=1}^{\infty} \sum_{t_1, \dots, t_n} P(t_1, \dots, t_n) \mathbb{1} \left[\begin{array}{l} P \sqsubseteq \text{decode}(t_1, \dots, t_n) \\ P \not\sqsubseteq \text{decode}(t_1, \dots, t_{n-1}) \end{array} \right] \\
 &\cong \sum_{n=1}^{\infty} \sum_{t_1, \dots, t_n} P(t_1, \dots, t_n) \mathbb{1} \left[\begin{array}{l} P \sqsubseteq \text{decode}(t_1, \dots, t_n) \\ P \not\sqsubseteq \text{decode}(t_1, \dots, t_{n-1}) \\ (t_1, \dots, t_n) \text{ is valid} \end{array} \right] \\
 &= \sum_{i=1}^{\ell} P(p_i^{(1)}, \dots, p_i^{(m_i)}).
 \end{aligned}$$

The last line follows from the definition of the Valid Covering Tree in Section 3. $\square$

The correctness of the completion sampling and byte-level sampling follow because every valid sequence that begins with the prefix must begin with a sequence from the Valid Covering Tree, and Proposition D.2 shows that we have properly accounted for every (valid) sequence that overlaps the prompt.

E. Extra Experimental Results

In this appendix, we report additional experimental results that did not fit in the main text.

E.1. Detailed Efficiency Comparison With Phan et al. (2025)

Comparing the efficiency of ByteSampler with that of Phan et al. (2025) is difficult because there is no model that is supported by both methods. In this section we make an approximate comparison using OLMo 2 7B with our method and Llama 2 7B with the method of Phan et al. (2025). These models have very similar size (7.30B and 6.74B respectively) and both use a standard dense transformer architecture without employing multi-query attention or grouped-query attention. Our benchmark setting is to sample 100 byte completions to questions from MMLU.

Table 9. Efficiency metrics for ByteSampler and the method of Phan et al. (2025). We report the total number of model inference tokens consumed by the method as well as the average number of bytes generated per second for each method. ByteSampler is significantly faster and requires fewer inference tokens. OLMo 2 7B is slightly larger than Llama 2 7B, so we expect ByteSampler to be at a slight disadvantage in this comparison.

Method	Model	Inference tokens	Throughput
Phan et al. (2025)	Llama 2 7B	637 $\pm$ 518	13.8 $\pm$ 0.4 bytes/sec
ByteSampler (ours)	OLMo 2 7B	190 $\pm$ 177	37.1 $\pm$ 0.8 bytes/sec

E.2. Detailed Comparison With Beam Summing Vieira et al. (2024)

We compare ByteSampler against the Beam Summing algorithm of Vieira et al. (2024). We evaluate on 10,000 document prefixes of 100 bytes sampled from the OLMo 2 training set. Because the beam summing implementation does not support OLMo 2, we use Llama 3 1B for all methods and run all methods on the same vLLM backend for a fair comparison.

Table 10. Byte-level cross-entropy, speed, and error rate for beam summing vs. ByteSampler.

Method	Loss (nats/byte)	Speed (bytes/sec)	Error rate (%)
ByteSampler	0.986 $\pm$ 0.003	620	0.0
Beam Summing (K = 1)	1.022 $\pm$ 0.003	153	25.6
Beam Summing (K = 3)	0.986 $\pm$ 0.003	118	1.47
Beam Summing (K = 10)	0.986 $\pm$ 0.003	48	1.42
Beam Summing (K = 128)	0.986 $\pm$ 0.003	4.7	1.42

Table 11. Speed comparison of ByteSampler and Beam Summing.

Regime	Method	Prompt bytes	Sampled bytes	Time (s)
Sampling heavy	ByteSampler	0	500	14.2
Sampling heavy	Beam Summing (K = 1)	0	500	13.1
Sampling heavy	Beam Summing (K = 3)	0	500	15.5
Sampling heavy	Beam Summing (K = 10)	0	500	21.1
Prompt heavy	ByteSampler	1000	100	2.95
Prompt heavy	Beam Summing (K = 1)	1000	100	25.4
Prompt heavy	Beam Summing (K = 3)	1000	100	29.6
Prompt heavy	Beam Summing (K = 10)	1000	100	35.2

Shown in the table, we find that ByteSampler is significantly faster than even a beam size of 1, while achieving similar loss to much larger beam sizes. It’s worth noting that the beam summing algorithm can fail when the beam becomes empty due to a lack of valid continuation tokens; this occurs for 25% of prefixes when using beam size = 1. We have recorded the error rate here, but the loss numbers are based on examples where no method failed.

E.3. Wall-Clock Overhead Compared to Standard Sampling

We also compare ByteSampler directly to standard token-level sampling on the same model. Using LLAMA-3.1-8B, we sample 512-byte completions from a 127-token / 636-byte prompt and collect 30 runs for each method.

Table 12. Wall-clock overhead relative to standard token-level sampling. “PBP mode” conditions on arbitrary byte prefixes while still sampling tokens, whereas “Byte Sampling” generates one byte at a time.

Method	Total time (s)	Sampling speed	Ratio
Normal sampling	2.85	182.7 ± 1.8	$1.00\times$
ByteSampler PBP Mode	2.84	182.5 ± 1.8	$1.00\times$
ByteSampler Byte Sampling	13.7	37.1 ± 0.2	$0.21\times$

These results show that correcting the prompt boundary problem alone introduces negligible wall-clock overhead. For exact byte-level sampling, the measured slowdown is close to the bytes-per-token factor, suggesting that ByteSampler is operating near the intrinsic cost of querying the model once per byte rather than once per token, with little additional overhead from the algorithm itself.

E.4. Probability Mass on Invalid Token Sequences

We measure the cross-entropy loss of the normal token-level model and compare it to the loss when invalid next tokens are masked out (local canonicalization, [Vieira et al. \(2025\)](#)), using the same dataset as in Table 3.

Table 13. Effect of masking invalid next tokens on cross-entropy. All columns are reported in nats/byte.

Metric	OLMo2 1B	OLMo2 7B
Token level	1.056142	0.792348
Local Canonicalization	1.055851	0.792140
Difference ($\times 10^{-4}$)	2.91 ± 0.18	2.08 ± 0.11

The results show that the probability mass assigned to invalid next tokens is small for both models, and that it decreases with model scale: the larger OLMo2 7B model shows a smaller gap between token-level and locally canonicalized loss than OLMo2 1B.

E.5. Code Completion

To demonstrate the importance of the prompt boundary problem to longer generative tasks, we measure performance on HumanEval Random Span (Bavarian et al., 2022), a FIM (“Fill in the Middle”) code-completion variant of HumanEval (Chen et al., 2021). The dataset is made of examples with a `prompt` and a `suffix`. The original task, given a `prompt` and `suffix`, is to generate a string `middle` such that the code `prompt + middle + suffix` will pass the tests. To make this task more suitable for our setup, we discard the `suffix` and ask the model to directly extend the `prompt` into a valid solution. In HumanEval Random Span, the prompt is selected to include the instructions for the code as well as a random prefix of the solution code itself, which makes this setting likely to exhibit prompt boundary issues. We report the results for Qwen3 1B in Table 14.

Table 14. HumanEval Random Span (prefix only) results for naive sampling, backtracking, and ByteSampler.

Method	pass@1
Naive	0.565
1 Token Backtracking (Token Healing)	0.716
1 Token Backtracking (Token Healing)	0.741
1 Token Backtracking (Token Healing)	0.738
ByteSampler (ours)	0.824

F. Advanced Decoding Methods

In Section 3, we focused on showing that our method is “exact.” To be precise, this means that sampling byte-wise using our method and sampling normally give exactly the same distributions of output text (modulo invalid token sequences, as we discussed in Section D). However, an important distinction arises when applying popular decoding techniques such as greedy decoding, top- k , top- p (Holtzman et al., 2020), or even temperatures other than 1. Applying these at the byte-level does not produce the same result as applying them at the token level. This is because these transformations have different effects when applied with different granularities (clearly, greedily selecting the most likely next byte is not the same as greedily selecting the most likely next token).

When sampling with ByteSampler, we can apply greedy decoding, top- k , top- p , and temperature, at the byte-level using the normal method. However, we can also apply it at the token level by transforming the logprobs of the Valid Covering Tree *prior* to the aggregation into byte probabilities. This preserves exactness in these settings and is able to support any kind of transform that can be applied to the log-probabilities produced by the model.

To explore the difference between these settings, we repeat the code completion experiments from Section E.5 using different sampling parameters.

Table 15. HumanEval Random Span (prefix only) results using ByteSampler with various sampling parameters

Sampling transform	Transform level	pass@1	Avg. completion length
Greedy	Byte	0.824	163 ± 191
Greedy	Token	0.820	165 ± 194
Top- $p = 0.95$	Byte	0.800	170 ± 193
Top- $p = 0.95$	Token	0.787	163 ± 186
Temperature = 0.7	Byte	0.810	164 ± 189
Temperature = 0.7	Token	0.790	162 ± 181

Interestingly, we find that the byte-level sampling transforms tend to slightly outperform the corresponding token-level sampling transforms. We think exploring the cause of this difference in performance is an interesting direction for future work.

G. Example Valid Covering Trees

Here we show complete Valid Covering Trees for several example prefixes. Unlike the tree in Fig. 2c, we show the actual tree as calculated by our algorithm. However, to allow them to fit on a page, we choose to display *only the internal nodes* of the tree (not the leaves). To denote where the hidden leaves would be, we display nodes that have leaves in **bold font**.

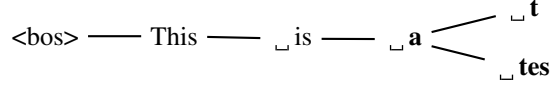


Figure 5. Example Valid Covering Tree for prefix “this is a tes” with the OLMo 2 tokenizer.

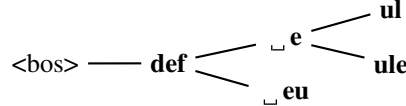


Figure 6. Example Valid Covering Tree for prefix “def eule” with the OLMo 2 tokenizer.

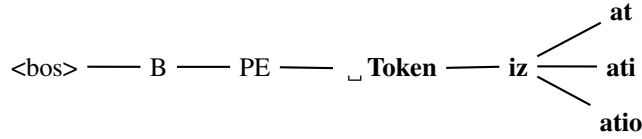


Figure 7. Example Valid Covering Tree for prefix “BPE Tokenizatio” with the OLMo 2 tokenizer.

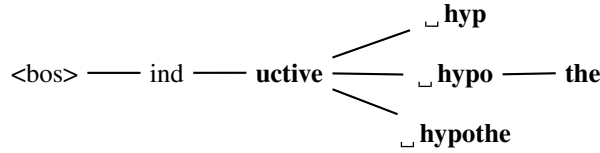


Figure 8. Example Valid Covering Tree for prefix “inductive hypothe” with the OLMo 2 tokenizer.

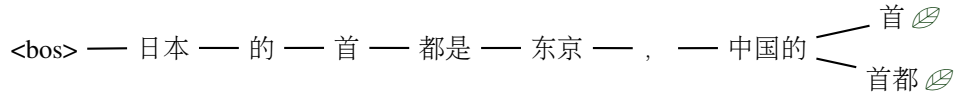


Figure 9. Example Valid Covering Tree for prefix “日本的首都是东京，中国的首都” with the QWEN3 tokenizer. We use  to denote nodes with leaves omitted.

We hide the leaves because it is typical for nodes that do have leaves to have dozens or even hundreds of them. To see how this can occur, imagine a prompt that ends on a space, and an internal node that ends right before that space. The node’s children will be all valid tokens that begin with a space. Most tokenizers have tens of thousands of tokens which begin with a space and nearly all of them will be valid continuations.

While this may sound problematic, we only need to query the next token distribution for the parent once in order to score all of its children, so this can be done efficiently in combination with the masking cache we describe in Section C.1.